

1. Block Programming

1.1 Concept of Block Programming

Block: A block is like a puzzle piece. Different blocks can be joined together to form a complete structure or program.

Programming: Programming is the process of giving instructions to a computer or electronic device so that it performs a required task.

Block Programming: Block programming is a visual programming approach in which ready-made, colorful blocks are joined together to create programs, games, animations, videos and other interactive projects. It is especially suitable for beginners and children because users build programs without typing traditional source code.

Note: Block programming reduces the need to remember programming syntax and lets learners concentrate on logic, creativity and problem solving.

1.2 Advantages of Block Programming

1. No syntax concern	Users do not need to worry about complex syntax or grammar rules used in text-based programming languages.
2. Focus on logic	More time can be spent on creativity, logical thinking and problem-solving skills.
3. Fewer typing errors	Since commands are selected and connected as blocks, common typing and spelling errors are reduced.
4. Visually appealing	Colorful blocks make programming engaging and easier for beginners and children to understand.
5. Easy transition to text	Block programming provides an interactive way to learn programming concepts before moving to text-based languages.

1.3 Applications of Block Programming

1. Educational Purpose
2. Gaming Development
3. Mobile Application Development
4. Robotics
5. Web Development
6. Project Creation
7. Modeling and Simulation

2. Introduction to Scratch

Scratch: Scratch is a popular block-based programming language and learning platform used to create interactive stories, animations and games. It was developed by the Lifelong Kindergarten Group at the MIT Media Lab and is provided free of charge.

2.1 Downloading Scratch on a PC

1. Open a web browser and visit: <https://scratch.mit.edu/download>
2. Download the Scratch installation file for your computer.
3. After the download is complete, open the installation file.
4. Follow the installation instructions and complete the setup.
5. Open the Scratch application after installation.
6. The Scratch interface will appear. The exact appearance may vary slightly with the version of Scratch being used.

Note: Scratch can also be used online through the Scratch website. The core project-building concepts are similar across the available versions.

2.2 Features of Scratch

Drag-and-drop interface	Programs are created by dragging blocks and placing them together.
Event-driven programming	Actions can begin when events occur, such as clicking the green flag or pressing a key.
Data and variables	Variables can store values such as score, time, names and other information.
Loops and conditions	Programs can repeat actions and make decisions based on conditions.
Sounds	Projects can include music, recorded sounds and sound effects.
Community sharing	Users can share projects and learn from projects created by other users.

3. Scratch Interface

Scratch has a colorful and engaging interface. Its main areas work together to allow the user to select sprites, choose blocks, edit the project and run the program.

3.1 Main Components of the Scratch Interface

Stage	The area where the project output, animation or game is displayed.
Sprite Block Palette	The area containing programming blocks grouped by category.
Sprite Area	The area where the sprites used in a project are listed and managed.
Extension Library	Provides additional block sets and device-related extensions.
Costumes Tab	Used to create, edit or change a sprite's appearance.
Sound Tab	Used to add, record, edit or manage sounds.
Sprite Info Panel	Shows and allows editing of information such as sprite name, position, size and direction.
File Tab / Menu	Provides options for creating, opening, saving and managing projects.
Choose a Sprite	Used to select or add a sprite to the project.
Choose a Backdrop	Used to select or add a background for the stage.
Project Name	Displays the name of the current Scratch project.
Green Flag	Starts or runs scripts connected to the green-flag event.
Backpack	Stores reusable code, costumes, sounds and other project items for easy reuse.

4. Concept of Blocks in Scratch

Block: A block is a colorful programming element that represents an action, command, event, condition or value. Blocks are connected together to form scripts that control sprites and the project.

Note: Different block categories use different colors, making them easy to identify and combine.

4.1 Main Scratch Block Categories

Motion Blocks	Control how a sprite moves, turns and changes position on the stage.
Loop / Control Blocks	Repeat actions, wait for a period of time, make decisions and control the flow of a program.
Sound Blocks	Play sounds, stop sounds and control sound effects or volume.
Event Blocks	Start scripts when events occur, such as clicking the green flag, pressing a key or receiving a message.
Control Blocks	Manage repetition, conditions, waiting and other program-control tasks.
Sensing Blocks	Detect inputs or conditions such as touching another sprite, mouse position or key presses.
Operators Blocks	Perform mathematical calculations, comparisons and logical operations.
Variables Blocks	Create and change variables, display values and store information such as scores.
My Blocks	Allow users to create their own custom blocks for reusable sets of instructions.

5. Demonstration: Dustbin Game Using Scratch

Stage 1: Plan the game	Decide the objective, rules, scoring system and how the player will control the game.
Stage 2: Set up the project	Create the Scratch project and choose a suitable backdrop for the game.
Stage 3: Add the sprites	Add the dustbin and waste-object sprites required for the game.
Stage 4: Program movement	Create scripts to move the dustbin or waste objects as required.
Stage 5: Detect collection	Use sensing and control logic to detect when waste reaches or touches the dustbin.
Stage 6: Add scoring	Create a score variable and increase or decrease it according to the game rules.
Stage 7: Add game control	Use events, conditions, sounds and messages to manage starting, playing and ending the game.
Stage 8: Test and improve	Run the game, find errors, correct the scripts and improve the project.

6. micro:bit

6.1 Concept and Definition of micro:bit

micro:bit: The BBC micro:bit is a small programmable computer designed to help learners create practical digital projects. It can be programmed to respond to buttons, sensors and other inputs and can control outputs such as the LED display and connected components.

6.2 History of micro:bit

2014	The BBC launched the Make It Digital initiative, which led to the development of the micro:bit project.
2015–2016	The BBC and partner organizations developed the micro:bit as a small, codeable computer for school-age learners.
2016	Around one million micro:bits were distributed to students across the UK as part of the education initiative.
2016 onward	The Micro:bit Educational Foundation was established to continue and expand the project internationally.
2020	An updated micro:bit version was introduced with additional features including a built-in microphone and speaker.

Note: The micro:bit project was designed to encourage young people to move from simply using technology to creating with technology.

6.3 Components of micro:bit

1. Microcontroller	The main processing unit that runs the program and controls the board.
2. 5 × 5 LED Matrix	A grid of 25 LEDs used to display letters, numbers, symbols and simple pictures.
3. Sensors	Inputs such as motion/accelerometer and compass sensing allow the board to detect changes in its environment.
4. Physical Components	The board contains electronic parts that support programming and interaction with the physical world.
5. Buttons	Physical buttons provide user input to the program.
6. Microphone	Detects sound levels and can be used in sound-responsive projects on supported versions.
7. Pin Connectors	Pins allow the micro:bit to connect with external sensors, LEDs, motors and other electronic components.

6.4 Applications of micro:bit

1. Education	Learning coding, electronics and physical computing.
2. Fitness	Building step counters, activity trackers and simple fitness projects.
3. Games	Creating simple physical or electronic games.
4. Fashion	Adding lights, sensors or interactive effects to wearable items.
5. Music	Creating sound-based projects and simple musical instruments.
6. Cooking	Building timers, alerts or sensor-based kitchen projects.
7. Home and Garden	Monitoring conditions or automating simple home and garden tasks.

7. Arduino Uno

7.1 Concept and Definition of Arduino Uno

Arduino Uno: Arduino Uno is a popular microcontroller development board used to build and test electronic projects. It can read inputs from sensors and other devices and control outputs such as LEDs, motors and other electronic components.

7.2 History of Arduino Uno

Period / Point	History
Early Arduino development	Arduino grew from classroom-focused experiments that aimed to make electronics development accessible and affordable for students, artists, designers and makers.
Uno milestone	The Arduino Uno became an important point in the Arduino product line, with “Uno” meaning “one” and marking a major step toward the version 1.0 era of the platform.
Growth	The board became widely used for education, prototyping, electronics projects and embedded systems because of its simple development environment and accessible hardware.

7.3 Components of Arduino Uno

Type	Meaning
Hardware	Physical parts of the board and connected electronic components.
Software	Programs and tools used to write, compile and upload code to the Arduino board.

7.4 Major Hardware Components

Hardware Part	Function
Microcontroller	The main chip that executes the uploaded program.
Digital I/O Pins	Used for digital input and output, such as reading switches or controlling LEDs.
Analog Input Pins	Read varying analog signals from sensors.
USB Port	Connects the board to a computer for programming and communication.
Power Connector	Provides power from an external source.
Reset Button	Restarts the program running on the board.
LED Indicators	Show power or board activity and can also be controlled in programs.

7.5 Software Components

Software Part	Function
Arduino IDE	The software environment used to write and upload programs to the board.
Sketch / Program	The code written by the user to define what the Arduino should do.
Libraries	Ready-made code that adds functions for sensors, displays, communication and other devices.

7.6 Applications of Arduino Uno

Application	Example Use
Education	Learning programming, electronics, sensors and microcontrollers.
Home Robotics	Building simple robots and automated home projects.
Wearable Technology	Creating smart wearable prototypes and interactive devices.
Data Gathering	Collecting readings from sensors for monitoring and analysis.
Gaming	Building physical controllers, interactive games and game prototypes.
Audio Projects	Creating simple sound, music and audio-control projects.
Testing	Prototyping and testing electronic circuits, sensors and control ideas.